

فاز ۲۰: مفاهیم پیشرفته Spring – Caching

Async, Scheduling, Validation

این بخش شامل مباحث پیشرفته‌تر Spring هست که توی پروژه‌های واقعی خیلی استفاده میشن.

۲۰.۱ Caching (کش کردن)

۲۰.۱.۱ فعال‌سازی Caching

```
1 @Configuration
2 @EnableCaching
3 public class CacheConfig {
4
5     @Bean
6     public CacheManager cacheManager() {
7         return new ConcurrentMapCacheManager("users", "orders", "products");
8     }
9 }
```

۲۰.۱.۲ @Cacheable (ذخیره در کش)

```
1 @Service
2 @Slf4j
3 public class UserService {
4
5     @Autowired
6     private UserRepository repository;
7
8     // ۱. ساده
9     @Cacheable("users")
10    public User findUser(Long id) {
11        log.info("Fetching user from database: {}", id);
12        return repository.findById(id).orElseThrow();
13    }
14
15    // ۲. با کلید سفارشی
16    @Cacheable(value = "users", key = "#username")
17    public User findByUsername(String username) {
18        return repository.findByUsername(username);
19    }
20
21    // ۳. با کلید ترکیبی
22    @Cacheable(value = "users", key = "#username + ':' + #password")
23    public User findByUsernameAndPassword(String username, String password) {
24        return repository.findByUsernameAndPassword(username, password);
25    }
26
```

```

27  // ۴. باشد  $id > 0$  فقط اگر (با شرط ۴)
28  @Cacheable(value = "users", condition = "#id > 0")
29  public User findUserWithCondition(Long id) {
30      return repository.findById(id).orElseThrow();
31  }
32
33  // ۵. (نداشتن) با شرط
34  @Cacheable(value = "users", unless = "#result == null")
35  public User findUserUnlessNull(Long id) {
36      return repository.findById(id).orElse(null);
37  }
38 }

```

۲۰.۱.۳ CacheEvict@ (پاک کردن کش)

```

1  @Service
2  public class UserService {
3
4      // ۱. پاک کردن یک کلید خاص
5      @CacheEvict(value = "users", key = "#id")
6      public void deleteUser(Long id) {
7          repository.deleteById(id);
8      }
9
10     // ۲. پاک کردن همه کش‌های مربوط
11     @CacheEvict(value = "users", allEntries = true)
12     public void clearAllCache() {

```

```

13    پاک میشن users همه کش های //
14    }
15
16    پاک کردن قبل از اجرای متد ۳. //
17    @CacheEvict(value = "users", key = "#id", beforeInvocation = true)
18    public void updateUser(User user) {
19        قبل از اجرا کش پاک میشه //
20        repository.save(user);
21    }
22 }

```

۲۰.۱.۴ CachePut@ (به روزرسانی کش)

```

1 @Service
2 public class UserService {
3
4    همیشه کش رو به روز می کنه (حتی اگه متد خطا بده) //
5    @CachePut(value = "users", key = "#user.id")
6    public User updateUser(User user) {
7        return repository.save(user);
8    }
9 }

```

۲۰.۱.۵ Redis با Caching

```

1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-data-redis</artifactId>
4 </dependency>
5 spring.redis.host=localhost
6 spring.redis.port=6379
7 spring.redis.password=
8 spring.redis.timeout=2000
9
10 spring.cache.type=redis
11 spring.cache.redis.time-to-live=600000 # ١٠ دقيقه
12 spring.cache.redis.cache-null-values=false
13 @Configuration
14 @EnableCaching
15 public class RedisCacheConfig {
16
17   @Bean
18   public RedisCacheConfiguration cacheConfiguration() {
19     return RedisCacheConfiguration.defaultCacheConfig()
20       .entryTtl(Duration.ofMinutes(10))
21       .disableCachingNullValues()
22       .serializeValuesWith(RedisSerializationContext.SerializationPair
23         .fromSerializer(new GenericJackson2JsonRedisSerializer()));
24   }
25 }

```

۲۰.۲ Async (پردازش ناهمگام)

۲۰.۲.۱ فعال سازی Async

```

1 @Configuration
2 @EnableAsync
3 public class AsyncConfig {
4
5     @Bean(name = "taskExecutor")
6     public Executor taskExecutor() {
7         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
8         executor.setCorePoolSize(5);    // های اولیه Thread
9         executor.setMaxPoolSize(10);    // ها حداکثر Thread
10        executor.setQueueCapacity(100); // صف انتظار
11        executor.setThreadNamePrefix("Async-");
12        executor.initialize();
13        return executor;
14    }
15}

```

۲۰.۲.۲ Async@

```

1 @Service
2 @Slf4j
3 public class EmailService {
4

```

```

5  // ۱. ساده
6  @Async
7  public void sendEmail(String to, String body) {
8      log.info("Sending email to: {} on thread: {}", to, Thread.currentThread().getName());
9      // کد ارسال ایمیل
10     try {
11         Thread.sleep(3000);
12     } catch (InterruptedException e) {
13         // ignore
14     }
15     log.info("Email sent to: {}", to);
16 }
17
18 // ۲. خاص Executor با
19 @Async("taskExecutor")
20 public void sendEmailWithCustomExecutor(String to, String body) {
21     // ...
22 }
23
24 // ۳. برگرداندن Future
25 @Async
26 public CompletableFuture<String> processData(String data) {
27     String result = data.toUpperCase();
28     return CompletableFuture.completedFuture(result);
29 }
30 }

```

۲۰.۲.۳ استفاده از Async

```
1 @RestController
2 @RequestMapping("/api/email")
3 public class EmailController {
4
5     @Autowired
6     private EmailService emailService;
7
8     @GetMapping("/send")
9     public ResponseEntity<String> sendEmail() {
10         emailService.sendEmail("test@example.com", "Hello!");
11         return ResponseEntity.ok("Email sending started");
12     }
13
14     @GetMapping("/process")
15     public CompletableFuture<ResponseEntity<String>> process() throws ExecutionException,
16     InterruptedException {
17         CompletableFuture<String> future = emailService.processData("hello");
18         String result = future.get();
19         return CompletableFuture.completedFuture(ResponseEntity.ok(result));
20 }
```

۲۰.۲.۴ مدیریت خطا در Async


```
1 @Configuration
2 @EnableAsync
3 public class AsyncConfig implements AsyncConfigurer {
4
5     @Override
6     public Executor getAsyncExecutor() {
7         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
8         executor.setCorePoolSize(5);
9         executor.setMaxPoolSize(10);
10        executor.setThreadNamePrefix("Async-");
11        executor.initialize();
12        return executor;
13    }
14
15    @Override
16    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
17        return (throwable, method, params) -> {
18            log.error("Async error in method: {}", method.getName(), throwable);
19        };
20    }
21 }
```

۲۰.۳ Scheduling (برنامه‌ریزی)

۲۰.۳.۱ فعال‌سازی Scheduling

```

1 @Configuration
2 @EnableScheduling
3 public class SchedulingConfig { ... }

```

۲۰.۳.۲ Scheduled@

```

1 @Component
2 @Slf4j
3 public class ScheduledTasks {
4
5     // ۱. با فاصله ثابت (بعد از اتمام کار قبلی) ۱.
6     @Scheduled(fixedDelay = 5000)
7     public void fixedDelayTask() {
8         log.info("Fixed delay task executed");
9     }
10
11     // ۲. با فاصله ثابت (بدون توجه به اتمام کار قبلی) ۲.
12     @Scheduled(fixedRate = 10000)
13     public void fixedRateTask() {
14         log.info("Fixed rate task executed");
15     }
16

```

```

17  // ۳. با تأخیر اولیه
18  @Scheduled(initialDelay = 10000, fixedRate = 30000)
19  public void delayedTask() {
20      log.info("Delayed task executed");
21  }
22
23  // ۴. با Cron Expression
24  @Scheduled(cron = "0 0 2 * *") // هر روز ساعت ۲ بامداد
25  public void cronTask() {
26      log.info("Cron task executed - daily backup");
27  }
28
29  // ۵. با Cron و TimeZone
30  @Scheduled(cron = "0 0 0 * * SAT", zone = "Asia/Tehran")
31  public void weeklyTask() {
32      log.info("Weekly task - every Saturday midnight Tehran time");
33  }
34 }

```

کامل Cron Expression ۲۰.۳.۳

```

1  _____ ثانیه (59-0)
2  | _____ دقیقه (59-0)
3  | | _____ ساعت (23-0)
4  | | | _____ روز ماه (31-1)
5  | | | | _____ ماه (یا 12-1 JAN-DEC)
6  | | | | | _____ روز هفته (یا 7-0 SUN-SAT)
7  | | | | |
8  * * * * *
9
10 مثالها:
11 "0 0 2 * * *" → هر روز ساعت ۲:۰۰ بامداد
12 "0 30 9 * * MON-FRI" → روزهای کاری ساعت ۹:۳۰ صبح
13 "0 0 0 1 * *" → اولین روز هر ماه ساعت ۱۲:۰۰
14 "0 0 12 * * 6" → هر جمعه ساعت ۱۲:۰۰
15 "0 0 0 * * 6" → هر جمعه نیمه شب
16 "0 0 0 1 1 *" → اول ژانویه نیمه شب
17 "0 0 0 * * 0" → هر یکشنبه نیمه شب
18 "0 0 0 15 * *" → پانزدهم هر ماه نیمه شب

```

۲۰.۳.۴ Scheduling با پروفایل

```

1 @Component
2 @Slf4j
3 @Profile("prod")
4 public class ProdScheduledTasks {
5
6     @Scheduled(cron = "0 0 2 * * *")
7     public void backupDatabase() {
8         log.info("Backing up database...");
9     }
10 }

```

۲۰.۴ Validation (اعتبارسنجی)

۲۰.۴.۱ فعال سازی Validation

```

1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-validation</artifactId>
4 </dependency>

```

۲۰.۴.۲ Annotation های پرکاربرد

```

1 @Data
2 public class UserRequest {
3
4     @NotNull(message = "ID نباید null باشد")
5     private Long id;

```

```

6
7  @NotBlank(message = "نام کاربری اجباری است")
8  @Size(min = 3, max = 20, message = "نام کاربری باید بین ۳ تا ۲۰ کاراکتر باشد")
9  private String username;
10
11  @NotBlank(message = "پسورد اجباری است")
12  @Size(min = 8, message = "پسورد باید حداقل ۸ کاراکتر باشد")
13  @Pattern(
14      regexp = "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z]).+$",
15      message = "پسورد باید شامل عدد و حروف بزرگ و کوچک باشد"
16  )
17  private String password;
18
19  @Email(message = "ایمیل نامعتبر است")
20  @NotBlank(message = "ایمیل اجباری است")
21  private String email;
22
23  @Min(value = 18, message = "سن باید حداقل ۱۸ باشد")
24  @Max(value = 100, message = "سن باید حداکثر ۱۰۰ باشد")
25  private Integer age;
26
27  @Pattern(
28      regexp = "^\\d{10}$",
29      message = "شماره تلفن باید ۱۰ رقم باشد"
30  )
31  private String phoneNumber;

```

```

32
33 @Size(max = 500, message = "توضیحات حداکثر ۵۰۰ کاراکتر")
34 private String description;
35 }

```

۲۰.۴.۳ استفاده در Controller

```

1 @RestController
2 @RequestMapping("/api/users")
3 public class UserController {
4
5     // ۱. @Valid اعتبارسنجی
6     @PostMapping
7     public ResponseEntity<User> createUser(@Valid @RequestBody UserRequest request) {
8         return ResponseEntity.ok(userService.save(request));
9     }
10
11     // ۲. @Validated با گروه‌ها
12     @PostMapping("/register")
13     public ResponseEntity<User> registerUser(@Validated @RequestBody UserRequest request) {
14         return ResponseEntity.ok(userService.register(request));
15     }
16
17     // ۳. BindingResult خطاهای اعتبارسنجی
18     @PostMapping("/validate")
19     public ResponseEntity<?> validateUser(@Valid @RequestBody UserRequest request,
20                                           BindingResult bindingResult) {

```

```

21     if (bindingResult.hasErrors()) {
22         List<String> errors = bindingResult.getAllErrors()
23             .stream()
24             .map(DefaultMessageSourceResolvable::getDefaultMessage)
25             .collect(Collectors.toList());
26         return ResponseEntity.badRequest().body(errors);
27     }
28     return ResponseEntity.ok("Valid");
29 }
30 }

```

۲۰.۴.۴ مدیریت خطاهای Validation (Global)

```

1 @ControllerAdvice
2 public class GlobalExceptionHandler {
3
4     @ExceptionHandler(MethodArgumentNotValidException.class)
5     public ResponseEntity<Map<String, List<String>>> handleValidationErrors(
6         MethodArgumentNotValidException ex) {
7         List<String> errors = ex.getBindingResult()
8             .getAllErrors()
9             .stream()
10             .map(DefaultMessageSourceResolvable::getDefaultMessage)
11             .collect(Collectors.toList());
12
13         return ResponseEntity.badRequest()
14             .body(Map.of("errors", errors));

```



```

15 }
16
17 @ExceptionHandler(ConstraintViolationException.class)
18 public ResponseEntity<Map<String, String>> handleConstraintViolation(
19     ConstraintViolationException ex) {
20     String message = ex.getConstraintViolations()
21         .stream()
22         .map(violation -> violation.getMessage())
23         .collect(Collectors.joining(", "));
24
25     return ResponseEntity.badRequest()
26         .body(Map.of("error", message));
27 }
28 }

```

۲۰.۴.۵ Group Validation (اعتبارسنجی گروهی)

```

1 // a. تعریف گروه ها
2 public interface Create {}
3 public interface Update {}
4
5 // b. استفاده در کلاس
6 @Data
7 public class UserRequest {
8     @NotNull(groups = Update.class)
9     private Long id;
10

```

```

11 @NotBlank(groups = {Create.class, Update.class})
12 private String username;
13
14 @NotBlank(groups = {Create.class})
15 private String password;
16 }
17
18 // c. استفاده در Controller
19 @PostMapping
20 public ResponseEntity<User> create(@Validated(Create.class) @RequestBody UserRequest request) {
21     // گروه Create فقط اعتبارسنجی
22 }
23
24 @PutMapping
25 public ResponseEntity<User> update(@Validated(Update.class) @RequestBody UserRequest request) {
26     // گروه Update فقط اعتبارسنجی
27 }

```

۲۰.۴.۶ Custom Validation (سفارشی)

```

1 //۱. Annotation
2 @Target({ElementType.FIELD, ElementType.PARAMETER})
3 @Retention(RetentionPolicy.RUNTIME)
4 @Constraint(validatedBy = IranNationalCodeValidator.class)
5 public @interface IranNationalCode {
6     String message() default "کد ملی نامعتبر است";
7     Class<?>[] groups() default {};

```

```

8  Class<? extends Payload>[] payload() default {};
9  }
10
11 // ۲. Validator
12 public class IranNationalCodeValidator implements
ConstraintValidator<IranNationalCode, String> {
13
14     @Override
15     public boolean isValid(String code, ConstraintValidatorContext context) {
16         if (code == null || code.length() != 10) {
17             return false;
18         }
19         // الگوریتم اعتبارسنجی کد ملی ایران
20         try {
21             // کد الگوریتم...
22             return true;
23         } catch (Exception e) {
24             return false;
25         }
26     }
27 }
28 // ۳. استفاده
29 @Data
30 public class PersonRequest {
31     @IranNationalCode
32     private String nationalCode;
33 }

```

۲۰.۵ ! سوالات مصاحبه‌ای

? سوال ۱: فرق @Cacheable و @CacheEvict چیست؟

جواب:

• @Cacheable: نتیجه متد رو در کش ذخیره می‌کنه

• @CacheEvict: کش رو پاک می‌کنه

? سوال ۲: @Async چه کاربردی داره؟

جواب: @Async برای اجرای ناهمگام (Asynchronous) متدهاست. یعنی متد در یک Thread جداگانه اجرا میشه و Thread اصلی منتظر نمی‌مونه.

? سوال ۳: Cron Expression چیست؟

جواب: یه فرمت برای مشخص کردن زمان اجرای کارهای زمان‌بندی شده. شامل ۶ بخش: ثانیه، دقیقه، ساعت، روز ماه، ماه، روز هفته.

? سوال ۴: Validation با @Valid و @Validated چه فرقی دارن؟

جواب:

• @Valid: از JPA میاد، برای اعتبارسنجی استاندارد

• @Validated: از Spring میاد، پشتیبانی از Group Validation داره

? سوال ۵: چطور کش رو در Spring فعال می‌کنیم؟

جواب: با اضافه کردن @EnableCaching به یک @Configuration کلاس.